Laboratório 07

Primeiro Contacto com Infrastructure as Code

Infraestruturas de Computação na Nuvem | Aula 07

Objetivo e enquadramento

A Aula 07 apresenta, no plano teórico, o princípio de Infrastructure as Code, a distinção entre abordagens declarativa e imperativa, a idempotência, a gestão de configuração com o Ansible e o provisionamento com o Terraform. Este laboratório torna esses conceitos observáveis numa máquina virtual construída no computador do próprio aluno, onde o Terraform cria containers, o Ansible os configura e o Ansible Vault protege um segredo.

O trabalho não requer acesso ao servidor da unidade curricular nem a qualquer serviço de nuvem. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**. Todo o trabalho decorre numa máquina virtual Ubuntu, criada e apagada com um único comando, de modo que o computador do aluno permanece intacto.

Nota

O Docker e o Terraform são mais simples de utilizar num núcleo Linux. Por esse motivo o laboratório utiliza uma máquina virtual criada com o `multipass`, que funciona da mesma forma em Windows, macOS e Linux. Quem utilizar Linux pode executar os passos diretamente no seu sistema, a partir do passo 1.3.

Resultados de aprendizagem

No final deste laboratório, o aluno deve ser capaz de:

- Distinguir uma operação imperativa de uma operação declarativa e idempotente.
- Escrever e executar um playbook Ansible com tasks, template e handler, e interpretar os resultados *changed* e *ok*.

- Observar o *drift* e a sua correção por uma nova execução do playbook.
- Usar o Terraform para criar recursos, ler o plano de execução e inspecionar o state file.
- Ligar as duas ferramentas, usando um inventory gerado pelo Terraform para configurar com o Ansible os containers criados.
- Cifrar um segredo com o Ansible Vault e verificar que não fica legível no repositório.

Requisitos

- Computador com Windows, macOS ou Linux.
- Cerca de 4 GB de memória livres para a máquina virtual e 15 GB de espaço em disco.
- Ligação à Internet, necessária para obter a imagem, os pacotes e os providers do Terraform.
- Permissão para instalar o `multipass` (Windows e macOS) ou o `snap` (Linux).
- Duração estimada: entre 90 e 120 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

Conteúdo do ficheiro `exemplo.txt`

```
Contedo a gravar num ficheiro, com um editor como o nano.
```

Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

Erro que pode ocorrer e respetiva resolução.

Nota

Informação complementar que facilita a execução.

Os comandos podem ser copiados e colados. Os resultados mostrados são exemplos: identificadores, tempos e números de versão diferem de computador para computador. Os ficheiros são criados em `~/icn-aula07`, dentro da máquina virtual.

Percurso

A figura seguinte resume a sequência do guião. Os passos 4 e 5 constituem a observação central: o Terraform cria os hosts e o Ansible configura-os, ligados por um inventory que o próprio Terraform gera.



Os passos 4 e 5 são a parte central: o Terraform cria os hosts e o Ansible configura-os.

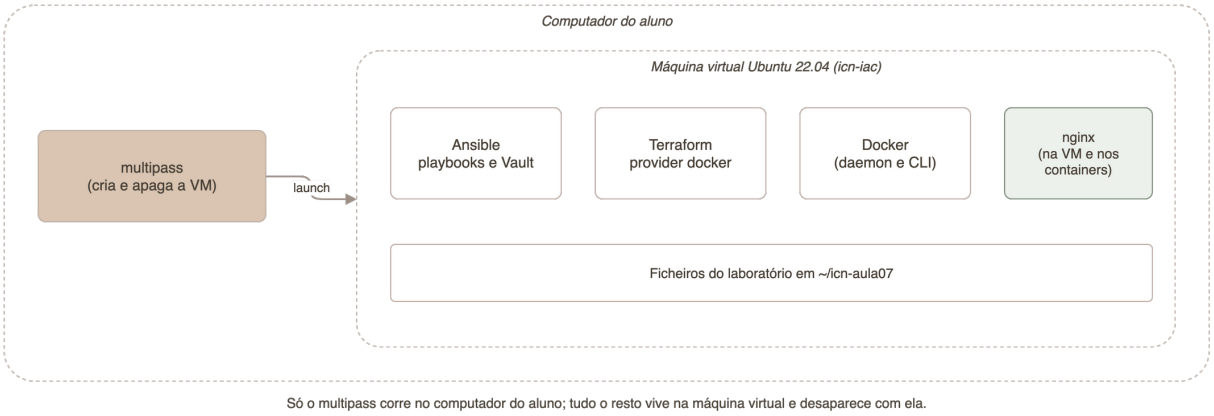
Como ler a figura

Os sete passos leem-se da esquerda para a direita. Os dois primeiros, a bege, preparam o ambiente e mostram a diferença entre executar passos e declarar um estado. O passo 3, a branco, usa o Ansible sozinho. Os passos 4 e 5, a verde, são a parte central: o Terraform provisiona os containers e o Ansible configura-os. O passo 6, a branco, acrescenta a proteção de um segredo. O passo 7, em tom avermelhado, encerra o trabalho e liberta os recursos. Abaixo de cada caixa, uma legenda resume o que é feito nesse passo.

Passo 1 — Preparação do ambiente

Objetivo. Criar uma máquina virtual Ubuntu e instalar nela o Ansible, o Docker e o Terraform.

Enquadramento. As três ferramentas representam camadas diferentes do que foi estudado na Aula 07. O Terraform provisiona recursos, o Ansible configura o que corre dentro deles e o Docker fornece os recursos que vão ser provisionados, sem exigir um servidor ou uma conta em nenhuma nuvem.



Só o multipass corre no computador do aluno; tudo o resto vive na máquina virtual e desaparece com ela.

Como ler a figura

A caixa exterior representa o computador do aluno. À esquerda está o `multipass`, o único programa instalado no computador, cuja função é criar e apagar a máquina virtual. A caixa interior é essa máquina virtual, onde estão as quatro peças do laboratório: o Ansible, o Terraform, o Docker e o nginx, que serve as páginas na máquina virtual e nos containers. Por baixo ficam os ficheiros do laboratório. Tudo o que é instalado fica dentro da máquina virtual e desaparece quando esta é apagada.

Fundamento teórico

O Ansible é *agentless*: não exige software residente nos hosts geridos, apenas uma forma de os alcançar. Normalmente essa forma é o SSH. Neste laboratório usam-se também a ligação local, para configurar a própria máquina virtual, e a ligação ao Docker, que executa os módulos dentro de um container com `docker exec`.

O Terraform, por seu lado, não sabe criar nada sozinho. Delega nos *providers*, plugins que traduzem o código em chamadas à API de cada plataforma. Aqui o provider usado é o do Docker, que fala com o daemon da máquina virtual.

Neste passo, deve analisar-se onde vive cada ferramenta. Apenas o `multipass` corre no computador do aluno, e tudo o resto corre dentro da máquina virtual.

1.1 Instalação do Multipass

Em macOS, com o Homebrew, executar o primeiro comando. Em Windows, utilizar o instalador disponível em https://multipass.run. Em Linux, executar o segundo comando.

Comando a executar

```
brew install --cask multipass
```

Comando a executar

```
sudo snap install multipass
```

Confirmar a instalação:

Comando a executar

```
multipass version
```

Resultado esperado (os valores podem variar)

```
multipass   1.xx.x
```

Erro possível e resolução

Se aparecer a mensagem `command not found`, fechar o terminal e abrir um novo. Em Windows, utilizar o PowerShell e confirmar que o instalador terminou sem erros.

1.2 Criação da máquina virtual

Comando a executar

```
multipass launch 22.04 --name icn-iac --cpus 2 --memory 4G --disk 15G
multipass shell icn-iac
```

Resultado esperado (os valores podem variar)

```
Launched: icn-iac
ubuntu@icn-iac:~$
```

A partir deste momento, todos os comandos são executados dentro da máquina virtual, no prompt `ubuntu@icn-iac`. A primeira execução demora alguns minutos, por incluir a obtenção da imagem.

Erro possível e resolução

Se o `launch` falhar ou ficar parado, o computador não dispõe de memória livre suficiente. Fechar outras aplicações e repetir. Se a máquina já existir, executar `multipass delete icn-iac`, `multipass purge` e repetir.

1.3 Instalação do Ansible e do Docker

Comando a executar

```
sudo apt update
sudo apt install -y ansible docker.io curl gnupg lsb-release
sudo usermod -aG docker ubuntu
```

O último comando permite ao utilizador `ubuntu` usar o Docker sem `sudo`, mas só produz efeito numa nova sessão. Sair da máquina virtual e entrar de novo:

Comando a executar

```
exit
multipass shell icn-iac
```

1.4 Instalação do Terraform

O Terraform é instalado a partir do repositório de pacotes da HashiCorp, de modo a obter sempre a versão estável mais recente.

Comando a executar

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share
    /keyrings/hashicorp-archive-keyring.gpg
```

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
    apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/
    sources.list.d/hashicorp.list
sudo apt update
sudo apt install -y terraform
```

1.5 Verificação

Comando a executar

```
ansible --version | head -1
docker version --format '{{.Server.Version}}'
terraform version | head -1
mkdir -p ~/icn-aula07 && cd ~/icn-aula07
```

Resultado esperado (os valores podem variar)

```
ansible [core 2.xx.x]
2x.x.x
Terraform v1.x.x
```

Erro possível e resolução

Se o Docker responder `permission denied`, a nova sessão não foi aberta. Sair da máquina virtual e entrar de novo com `multipass shell icn-iac`, e confirmar com o comando `groups` que o grupo `docker` aparece na lista.

Verificação antes de avançar

Os três comandos de versão executam sem erro e o diretório `~/icn-aula07` existe.

Passo 2 — Imperativo e declarativo

Objetivo. Comparar a execução repetida de um script com a execução repetida de um módulo Ansible, e observar a idempotência.

Enquadramento. A Aula 07 distingue descrever os passos a executar de descrever o estado pretendido. Este passo reproduz, em ponto pequeno, o exemplo da aula: acrescentar uma linha a um ficheiro de configuração.

Fundamento teórico

Num script, cada execução repete os passos, independentemente do que já foi feito. Acrescentar uma linha a um ficheiro com » duplica-a de cada vez que o script corre. Um módulo declarativo, como o `lineinfile` do Ansible, compara o estado atual com o pretendido e só atua se houver diferença.

Os módulos reportam o resultado de cada execução. O valor *changed* indica que o estado foi alterado e *ok* indica que já estava conforme. É esta informação que torna as execuções auditáveis.

Neste passo, deve analisar-se quantas linhas tem o ficheiro depois de duas execuções de cada abordagem.

2.1 Abordagem imperativa

Conteúdo do ficheiro `imperativo.sh`

```
#!/bin/bash
echo "porta=8080" >> imperativo.conf
```

Comando a executar

```
cd ~/icn-aula07
bash imperativo.sh
bash imperativo.sh
cat imperativo.conf
```

Resultado esperado (os valores podem variar)

```
porta=8080
porta=8080
```

2.2 Abordagem declarativa

Comando a executar

```
ansible localhost -c local -m ansible.builtin.lineinfile -a "path=/home/ubuntu/icn
    -aula07/declarativo.conf line=porta=8080 create=yes"
ansible localhost -c local -m ansible.builtin.lineinfile -a "path=/home/ubuntu/icn
    -aula07/declarativo.conf line=porta=8080 create=yes"
cat declarativo.conf
```

Resultado esperado (os valores podem variar)

```
localhost | CHANGED => { ... "changed": true, ... }
localhost | SUCCESS => { ... "changed": false, ... }
porta=8080
```

Analisar

O ficheiro imperativo tem duas linhas e o declarativo tem uma. Na segunda execução do Ansible o resultado passou de *CHANGED* para *SUCCESS* com `"changed": false`, o que significa que nada foi alterado. Em que situação seria perigoso executar duas vezes o script imperativo, por exemplo num ficheiro de configuração de um serviço?

Verificação antes de avançar

O ficheiro `imperativo.conf` tem duas linhas e o ficheiro `declarativo.conf` tem apenas uma.

Passo 3 — O primeiro playbook

Objetivo. Escrever um playbook que instala o nginx na máquina virtual, gera uma página a partir de um template e reinicia o serviço apenas quando necessário. Executá-lo duas vezes e provocar *drift*.

Enquadramento. O playbook usa os componentes estudados na aula: tasks executadas por módulos, um template Jinja2 preenchido com *facts* e um handler que só corre quando uma task notifica uma alteração.

Fundamento teórico

Um playbook é um ficheiro YAML que associa tasks a hosts. Cada task invoca um módulo, e os módulos do Ansible são idempotentes: verificam o estado antes de agir. O template Jinja2 é preenchido com variáveis e com os facts que o Ansible recolhe sobre o host, como o sistema operativo ou a memória.

O handler distingue-se de uma task normal por só ser executado quando alguém o notifica, e uma só vez no final, mesmo que várias tasks o tenham notificado. É o mecanismo que evita reiniciar um serviço sem necessidade.

Neste passo, deve analisar-se o que muda entre a primeira e a segunda execução, e o que acontece quando o estado real deixa de coincidir com o declarado.

3.1 Template e playbook

Conteúdo do ficheiro `index.html.j2`

```
<h1>Aula 07 - IaC</h1>
<p>Servidor {{ inventory_hostname }} com {{ ansible_distribution }} {{
    ansible_distribution_version }}</p>
<p>Memria total: {{ ansible_memtotal_mb }} MB</p>
```

Conteúdo do ficheiro `local.yml`

```
- name: Configurar o nginx na maquina virtual
  hosts: localhost
  connection: local
  become: true
  tasks:
    - name: Instalar o nginx
      ansible.builtin.apt:
```

```
        name: nginx
        state: present
        update_cache: true
    - name: Gerar a pagina inicial a partir do template
      ansible.builtin.template:
        src: index.html.j2
        dest: /var/www/html/index.nginx-debian.html
        mode: "0644"
      notify: Reiniciar o nginx
    - name: Garantir que o servico esta ativo
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
  handlers:
    - name: Reiniciar o nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

Nota

O YAML é sensível à indentação e não aceita tabulações. Cada nível usa dois espaços, e os ficheiros devem ser guardados exatamente como mostrado.

3.2 Primeira execução

Comando a executar

```
cd ~/icn-aula07
ansible-playbook local.yml
curl -s localhost
```

Resultado esperado (os valores podem variar)

```
TASK [Instalar o nginx] ... changed: [localhost]
TASK [Gerar a pagina inicial a partir do template] ... changed: [localhost]
RUNNING HANDLER [Reiniciar o nginx] ... changed: [localhost]
PLAY RECAP: localhost : ok=5  changed=3  unreachable=0  failed=0
<h1>Aula 07 - IaC</h1>
```

3.3 Segunda execução

Comando a executar

```
ansible-playbook local.yml
```

Resultado esperado (os valores podem variar)

```
PLAY RECAP: localhost : ok=4  changed=0  unreachable=0  failed=0
```

Analisar

Na primeira execução houve alterações e o handler correu. Na segunda, tudo estava conforme (*changed=0*) e o handler não foi executado. O código descreve o estado, e não os passos, por isso repetir a execução é seguro.

3.4 Drift

Simular uma alteração manual, que foge ao controlo do código, e verificar o estado:

Comando a executar

```
echo "alterado a mao" | sudo tee /var/www/html/index.nginx-debian.html
curl -s localhost
```

Antes de reaplicar, pedir ao Ansible que mostre o que faria, sem alterar nada:

Comando a executar

```
ansible-playbook local.yml --check --diff
```

Resultado esperado (os valores podem variar)

```
TASK [Gerar a pagina inicial a partir do template]
--- before: alterado a mao
+++ after: <h1>Aula 07 - IaC</h1> ...
changed: [localhost]
```

Reaplicar e confirmar que a página volta ao estado declarado:

Comando a executar

```
ansible-playbook local.yml
curl -s localhost
```

Analisar

A opção `--check --diff` mostra a diferença entre o estado real e o declarado, sem a aplicar. Esta é a mesma ideia do `terraform plan`, que será usado no passo seguinte. Ao reaplicar, o Ansible repôs a página e reiniciou o serviço, porque o drift foi detetado.

Verificação antes de avançar

A segunda execução reporta *changed=0* e, depois do drift, a página volta a mostrar o conteúdo do template.

Passo 4 — Provisionar com o Terraform

Objetivo. Descrever em código dois containers, ler o plano de execução, aplicá-lo e inspecionar o state file. Observar também como o Terraform reage a uma alteração manual.

Enquadramento. No passo anterior, o Ansible configurou uma máquina que já existia. Aqui a pergunta é outra: como se garante que as máquinas existem? É o problema do provisionamento, e os containers fazem o papel de servidores a criar.

Fundamento teórico

Um ficheiro `.tf` descreve o estado desejado em HCL. O comando `terraform init` obtém os providers necessários. O `terraform plan` compara o código com o state file e com o estado real, e mostra o que seria criado, alterado ou destruído. Só o `terraform apply` executa as alterações, depois de confirmação.

O state file regista o mapeamento entre os recursos do código e os recursos reais. É por consultar este ficheiro que o Terraform sabe distinguir um recurso já criado de um que falta, e é por isso que ele é tão importante, e sensível, em trabalho de equipa.

Neste laboratório, o recurso `docker_container` usa `count`: o número de containers é uma variável, e alterá-la é a forma declarativa de escalar.

Neste passo, deve analisar-se a diferença entre o plano e a aplicação, e o conteúdo do state file.

4.1 O ficheiro main.tf

Comando a executar

```
mkdir -p ~/icn-aula07/terraform
cd ~/icn-aula07/terraform
nano main.tf
```

Conteúdo do ficheiro `main.tf`

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
```

```
    local = {
      source = "hashicorp/local"
    }
  }
}

provider "docker" {}

variable "replicas" {
  type    = number
  default = 2
}

resource "docker_image" "base" {
  name = "debian:12-slim"
}

resource "docker_container" "web" {
  count   = var.replicas
  name    = "web-${count.index + 1}"
  image   = docker_image.base.image_id
  command = ["sleep", "infinity"]

  ports {
    internal = 80
    external = 8081 + count.index
  }
}
```

4.2 Inicialização e plano

Comando a executar

```
terraform init
terraform plan
```

Resultado esperado (os valores podem variar)

```
Terraform has been successfully initialized!
...
  # docker_container.web[0] will be created
  # docker_container.web[1] will be created
  # docker_image.base will be created
```

```
Plan: 3 to add, 0 to change, 0 to destroy.
```

Analisar

O plano não alterou nada: apenas descreve o que seria feito. Contar quantos recursos serão criados e relacionar com o código. Porque é que o `docker_image` aparece, se apenas foram pedidos dois containers?

4.3 Aplicação

Comando a executar

```
terraform apply
```

Escrever `yes` quando o Terraform pedir confirmação.

Resultado esperado (os valores podem variar)

```
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Comando a executar

```
docker ps --format 'table {{.Names}}\t{{.Ports}}'
terraform state list
```

Resultado esperado (os valores podem variar)

```
NAMES   PORTS
web-2   0.0.0.0:8082->80/tcp
web-1   0.0.0.0:8081->80/tcp
docker_container.web[0]
docker_container.web[1]
docker_image.base
```

Erro possível e resolução

Se o `apply` falhar com `permission denied` no socket do Docker, a sessão não tem o grupo `docker`. Sair da máquina virtual, entrar de novo e repetir. Se falhar por o porto 8081 estar

ocupado, identificar o processo com `sudo ss -ltnp` e terminá-lo.

4.4 Escalar por alteração do código

Alterar o número de replicas e pedir apenas o plano:

Comando a executar

```
terraform plan -var replicas=3
```

Resultado esperado (os valores podem variar)

```
  # docker_container.web[2] will be created
Plan: 1 to add, 0 to change, 0 to destroy.
```

Analisar

O Terraform compara o estado desejado (três containers) com o state file (dois) e calcula apenas a diferença: um recurso a criar. Não foi necessário descrever como criar o terceiro container.

4.5 Drift

Remover um container fora do Terraform e deixar que o plano o detete:

Comando a executar

```
docker rm -f web-2
terraform plan
```

Resultado esperado (os valores podem variar)

```
  # docker_container.web[1] will be created
Plan: 1 to add, 0 to change, 0 to destroy.
```

Comando a executar

```
terraform apply -auto-approve
docker ps --format '{{.Names}}'
```

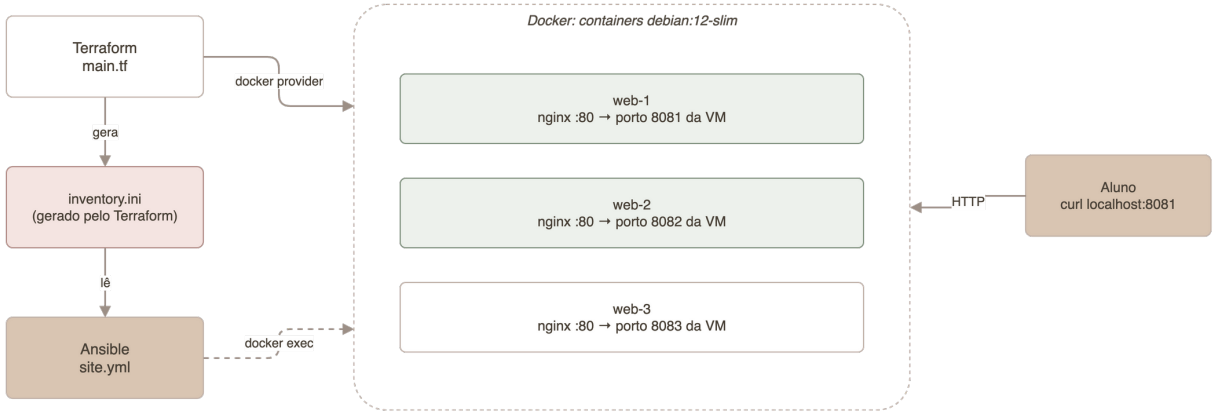
Verificação antes de avançar

Depois do `apply`, os containers `web-1` e `web-2` existem de novo, e o `terraform plan` indica *No changes*.

Passo 5 — Do Terraform ao Ansible

Objetivo. Fazer com que o Terraform gere o inventory do Ansible, configurar com o Ansible os containers criados e observar o que acontece quando se aumenta o número de replicas.

Enquadramento. Este é o passo que liga as duas camadas estudadas na aula. O Terraform decide que hosts existem e o Ansible decide o que corre dentro deles. A ponte entre ambos é um ficheiro, o inventory, que o primeiro escreve e o segundo lê.



O web-3 só existe depois de aumentar o número de replicas no Terraform (passo 5).

Como ler a figura

À esquerda estão as três peças de código. O Terraform, em cima, cria os containers através do provider do Docker e gera o ficheiro `inventory.ini`. O Ansible, em baixo, lê esse inventory e executa o playbook em cada container com `docker exec`, representado a tracejado. A caixa central contém os containers: o `web-1` e o `web-2` a verde existem desde o passo 4, e o `web-3` a branco só passa a existir quando se aumentar o número de replicas. À direita, o aluno testa cada servidor com `curl` através do porto correspondente da máquina virtual.

Fundamento teórico

Um container novo não tem Python, e o Ansible precisa dele no host para executar a maioria dos módulos. Esta é a contrapartida do modelo agentless: não há agente, mas há requisitos mínimos. O primeiro play usa o módulo `raw`, que executa um comando sem depender de Python, para instalar o interpretador. Só depois se usam os módulos habituais.

A ligação ao container usa o *connection plugin* `community.docker.docker`, em vez do SSH. O resto do Ansible é igual: inventory, playbook, tasks e handlers. Isto mostra que o Ansible é independente do transporte.

Neste passo, deve analisar-se quais os hosts que o Ansible altera quando o Terraform

acrescenta um container.

5.1 O inventory gerado pelo Terraform

Acrescentar ao fim do `main.tf` o recurso que gera o inventory e uma saída com os endereços:

Conteúdo do ficheiro `main.tf` (acrescentar ao fim)

```
resource "local_file" "inventory" {
  filename = "${path.module}/inventory.ini"
  content  = templatefile("${path.module}/inventory.tftpl", {
    nomes = docker_container.web[*].name
  })
}

output "urls" {
  value = [for c in docker_container.web : "http://localhost:${c.ports[0].external
    }"]
}
```

Conteúdo do ficheiro `inventory.tftpl`

```
[web]
%{ for nome in nomes ~}
${nome} ansible_connection=community.docker.docker
%{ endfor ~}
```

Comando a executar

```
terraform init
terraform apply -auto-approve
cat inventory.ini
```

Resultado esperado (os valores podem variar)

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
urls = ["http://localhost:8081", "http://localhost:8082"]

[web]
web-1 ansible_connection=community.docker.docker
web-2 ansible_connection=community.docker.docker
```

5.2 O playbook para os containers

Conteúdo do ficheiro `ansible.cfg`

```
[defaults]
inventory = terraform/inventory.ini
host_key_checking = False
interpreter_python = auto_silent
```

Conteúdo do ficheiro `site.yml`

```
- name: Preparar os hosts (Python)
  hosts: web
  gather_facts: false
  tasks:
    - name: Instalar o Python se nao existir
      ansible.builtin.raw: >
        test -x /usr/bin/python3 ||
        (apt-get update -qq && apt-get install -y -qq python3 python3-apt)
      changed_when: false

- name: Configurar os servidores web
  hosts: web
  tasks:
    - name: Instalar o nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600
    - name: Gerar a pagina inicial a partir do template
      ansible.builtin.template:
        src: index.html.j2
        dest: /var/www/html/index.nginx-debian.html
        mode: "0644"
      notify: Recarregar o nginx
    - name: Arrancar o nginx
      ansible.builtin.command: nginx
      args:
        creates: /run/nginx.pid
  handlers:
    - name: Recarregar o nginx
      ansible.builtin.command: nginx -s reload
```

Nota

Os ficheiros `ansible.cfg`, `site.yml` e `index.html.j2` ficam em `~/icn-aula07`, e o comando `ansible-playbook` deve ser executado a partir desse diretório. O `main.tf` e o inventory ficam em `~/icn-aula07/terraform`.

5.3 Execução

Comando a executar

```
cd ~/icn-aula07
ansible-playbook site.yml
curl -s localhost:8081
curl -s localhost:8082
```

Resultado esperado (os valores podem variar)

```
PLAY RECAP
web-1 : ok=4  changed=3  unreachable=0  failed=0
web-2 : ok=4  changed=3  unreachable=0  failed=0
<h1>Aula 07 - IaC</h1>
<p>Servidor web-2 com Debian 12</p>
```

Erro possível e resolução

Se o Ansible indicar que a ligação ao container falhou, confirmar que o nome no inventory coincide com o resultado de `docker ps`. Se aparecer uma mensagem sobre a coleção `community.docker`, instalar com `ansible-galaxy collection install community.docker`.

5.4 Escalar e reconfigurar

Aumentar o número de replicas no Terraform, regenerar o inventory e voltar a executar o playbook:

Comando a executar

```
cd ~/icn-aula07/terraform
terraform apply -auto-approve -var replicas=3
cd ~/icn-aula07
ansible-playbook site.yml
curl -s localhost:8083
```

Resultado esperado (os valores podem variar)

```
web-1 : ok=3  changed=0  unreachable=0  failed=0
web-2 : ok=3  changed=0  unreachable=0  failed=0
web-3 : ok=4  changed=3  unreachable=0  failed=0
<p>Servidor web-3 com Debian 12</p>
```

Analisar

O `web-1` e o `web-2` reportam *changed=0*: já estavam conforme, e o Ansible não lhes tocou. Só o `web-3`, novo, foi configurado. A mudança de infraestrutura foi feita num valor (`replicas`), e a configuração seguiu-a sem alterar o playbook. Qual seria o trabalho equivalente se os servidores fossem configurados à mão?

Verificação antes de avançar

O `curl` responde nos três portos (8081, 8082 e 8083) com a página gerada pelo template, e a segunda execução do playbook não alterou os servidores já existentes.

Passo 6 — Gestão de segredos com o Ansible Vault

Objetivo. Cifrar uma credencial com o Ansible Vault, verificar que não fica legível no ficheiro e usá-la num playbook sem a expor.

Enquadramento. O código de infraestrutura vai para o Git, mas as passwords não podem ir com ele em texto simples. A aula apresentou o Ansible Vault como uma das soluções, e este passo mostra o seu funcionamento.

Fundamento teórico

O Ansible Vault cifra ficheiros de variáveis com uma palavra-passe. O ficheiro cifrado pode ser guardado no repositório, porque o seu conteúdo só é legível por quem conhece a palavra-passe. Na execução, o Ansible pede a palavra-passe, decifra em memória e usa as variáveis como quaisquer outras.

Numa task que manipula o segredo, a opção `no_log` impede que o valor apareça na saída do Ansible e nos registos.

Neste passo, deve analisar-se o aspeto do ficheiro cifrado e o que o Ansible mostra, e não mostra, durante a execução.

6.1 Criação do ficheiro cifrado

Comando a executar

```
cd ~/icn-aula07
ansible-vault create segredos.yml
```

O Ansible pede uma palavra-passe para o cofre, que deve ser fornecida duas vezes (neste laboratório pode usar-se uma palavra-passe simples, por exemplo `icn2026`). Abre-se depois um editor, onde se escreve:

Conteúdo do ficheiro `segredos.yml` (dentro do editor)

```
db_password: "palavra-passe-da-base-de-dados"
```

Guardar e sair do editor. Observar o conteúdo do ficheiro tal como fica no disco:

Comando a executar

```
cat segredos.yml
```

Resultado esperado (os valores podem variar)

```
$ANSIBLE_VAULT;1.1;AES256
6338613161633531...
```

6.2 Utilização no playbook

Conteúdo do ficheiro `segredo.yml`

```
- name: Usar um segredo cifrado
  hosts: web
  gather_facts: false
  vars_files:
    - segredos.yml
  tasks:
    - name: Gravar o segredo no container
      ansible.builtin.copy:
        content: "{{ db_password }}"
        dest: /etc/app-secret
        mode: "0600"
      no_log: true
```

Comando a executar

```
ansible-playbook segredo.yml --ask-vault-pass
docker exec web-1 cat /etc/app-secret
```

Resultado esperado (os valores podem variar)

```
Vault password:
TASK [Gravar o segredo no container] ... changed: [web-1]
palavra-passe-da-base-de-dados
```

Erro possível e resolução

Se a execução falhar com `Attempting to decrypt but no vault secrets found`, faltou a opção `--ask-vault-pass`. Se falhar com `Decryption failed`, a palavra-passe introduzida é diferente da usada na criação do ficheiro.

Analisar

O ficheiro `segredos.yml` pode ser guardado no repositório, porque o conteúdo é ilegível sem a palavra-passe. A saída do playbook não mostra o valor do segredo, graças ao `no_log`. Num cenário real, a palavra-passe do cofre seria injetada pelo sistema de CI/CD e nunca escrita no repositório.

Verificação antes de avançar

O ficheiro `segredos.yml` começa por `$ANSIBLE_VAULT` e o container contém o segredo em `/etc/app-secret`.

Passo 7 — Limpeza

Objetivo. Destruir os recursos criados e apagar a máquina virtual, libertando memória e disco.

Enquadramento. Tal como a criação, a destruição é declarativa: o Terraform conhece tudo o que criou, através do state file, e remove exatamente isso.

7.1 Destruição dos recursos

Comando a executar

```
cd ~/icn-aula07/terraform
terraform destroy -auto-approve
docker ps -a
```

Resultado esperado (os valores podem variar)

```
Destroy complete! Resources: 5 destroyed.
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

7.2 Eliminação da máquina virtual

Sair da máquina virtual e, no computador do aluno, apagá-la:

Comando a executar

```
exit
multipass delete icn-iac
multipass purge
multipass list
```

Resultado esperado (os valores podem variar)

```
No instances found.
```

Verificação antes de avançar

O comando `multipass list` não apresenta a instância `icn-iac`.

Problemas frequentes

Sintoma	Resolução
`permission denied` no Docker	Sair e entrar de novo na máquina virtual para ativar o grupo `docker`.
YAML rejeitado pelo Ansible	Substituir tabulações por espaços e rever a indentação.
`terraform init` falha	Verificar a ligação à Internet e repetir.
Porto 8081 ocupado	Identificar o processo com `sudo ss -ltnp` e terminá-lo.
Ansible sem Python no container	Executar primeiro o play de preparação do `site.yml`.
Esqueceu a palavra-passe do cofre	Criar novo `segredos.yml`; o ficheiro cifrado não é recuperável.

Comandos utilizados

Comando	Finalidade
`multipass launch 22.04 --name icn-iac`	Criar a máquina virtual
`ansible localhost -m lineinfile`	Executar um módulo isolado (ad hoc)
`ansible-playbook playbook.yml`	Executar um playbook
`ansible-playbook --check --diff`	Mostrar as alterações sem as aplicar
`ansible-vault create ficheiro.yml`	Criar um ficheiro cifrado
`terraform init`	Obter os providers
`terraform plan`	Mostrar as alterações previstas
`terraform apply`	Executar as alterações
`terraform state list`	Listar os recursos no state file
`terraform destroy`	Destruir os recursos
`docker ps`	Listar os containers em execução
`multipass delete icn-iac`	Apagar a máquina virtual

Questões de consolidação

As questões seguintes destinam-se a discussão e não têm de ser entregues.

1. Porque é que o ficheiro imperativo ficou com duas linhas e o declarativo com uma? Que propriedade explica a diferença?

2. O que distingue o resultado *changed* do resultado *ok*, e porque é que é útil conhecê-lo?

3. Em que se assemelham o `terraform plan` e o `ansible-playbook --check --diff`? Em que diferem?

4. Qual é o papel do state file? O que aconteceria ao Terraform se o ficheiro fosse apagado?

5. Porque é que, ao passar de dois para três containers, o Ansible só alterou o `web-3`?

6. Porque é que o ficheiro `segredos.yml` pode ser guardado no Git, e que cuidado exige a palavra-passe do cofre?

Síntese

O laboratório permitiu observar, na prática, as ideias centrais da Aula 07:

- Um script **imperativo** repete os passos, e um módulo **declarativo** só atua quando há diferença: é a **idempotência**.
- O Ansible **configura** o que corre dentro de uma máquina e distingue o que mudou (*changed*) do que já estava conforme (*ok*).
- O **drift** é detetado e corrigido por nova execução, tanto no Ansible como no Terraform.
- O Terraform **provisiona** recursos, mostra o plano antes de o aplicar e regista o resultado no **state file**.
- As duas camadas ligam-se por um **inventory** gerado pelo Terraform: alterar um valor no código basta para escalar e reconfigurar.
- Um **segredo** cifrado com o Ansible Vault pode viver no repositório sem ficar exposto.